课程笔记

软件仓库管理（二）：为 Agent 重新设计版本管理

南京大学《生成式软件工程》（2026 秋）课程笔记

lecture-to-notes 整理

2026 年 9 月 16 日



视频作者/频道：绿导师原谅你了（Bilibili）
发布日期：2026-09-15
视频时长：98 分 29 秒
视频链接：https://www.bilibili.com/video/BV1Q6en6NEUo/

目录

1 为什么实验从“一个会成长的个人助手”开始 3

1.1 软件项目的三层：intent、spec、code 4

1.2 本章小结 4

2 文件系统的原罪：可追踪性与软件形态 5

2.1 技术债如何产生：一个 linked list 的例子 5

2.2 如果重新设计：repo.db 与“用魔法打败魔法” 6

2.3 本章小结 7

3 Git 的模型：快照、merge 与 rebase 7

3.1 merge 的隐藏问题：作者归属被改写 8

3.2 rebase：把修改重放到新的起点 8

3.3 我们真正想操作的是什么? 8

3.4 本章小结 10

4 把 Change 作为一等公民：Jujutsu 的设计 10

4.1 change ID 延续，commit ID 会变 10

4.2 连 staging area 都不需要了 11

4.3 肌肉记忆与学习的方向 12

4.4 本章小结 12

5 为 Agent 重新设计：从单线程人类到 Agent 团队 12

5.1 所有 VCS 的共同假设：使用者是人类 13

5.2 Agent 提速之后，Git 跟不上 13

5.3 Workaround 一：每个 Agent 一台开发机 14

5.4 Workaround 二：git worktree 14

5.5 本章小结 16

6 并发控制的新方向：从乐观到悲观 16

6.1 我们还需要 git blame 吗? 16

6.2 Git 其实是乐观并发控制 17

6.3 Agent 更快之后：换用悲观并发控制 17

6.4 本章小结 18

7 工具之上：团队管理与“斩杀线” 18

7.1 AI 是一条“斩杀线” 19

7.2 团队的复杂性：平方级的沟通关系 19

7.3 版本控制留给我们的启发 20

7.4 本章小结 21

8 从零开始一个项目：以不变应万变 21

8.1 你就是包工头：从零开始的难度 21

8.2 一个岔开的话题：汉诺塔与 Pico Playground 21

8.3 GPT 的“正常”设计 vs 版本管理 22
8.4 现场演示：直接丢需求会得到什么 23
8.5 MVP：先验证最值得验证的假设 24
8.6 早期的 Prompt，影响最大 25
8.7 本章小结 26

9 总结与延伸 **26**
9.1 讲者的收尾 26
9.2 本讲义的提炼 26
9.3 拓展阅读 27

1 为什么实验从“一个会成长的个人助手”开始

这一讲是上一讲“软件仓库管理（一）”的延续。开场先布置了本课程第一个 Lab：**会成长的个人助手**。按照课程一贯的说法，所有作业都是选做、自愿完成，最后验收时由讲师的机器人给提交链接评分。这个 Lab 对实现方式完全不设限：语言、模型和交互形式都可以自己选，甚至鼓励直接让 Agent 去实现。

Lab 1 的五项基础能力

- **个人数据库**：能索引、检索你的资料，并随资料变化更新索引，回答时能定位到原文。格式不限，推荐 Markdown 加目录结构——你可以直接阅读、修改文件，也更容易观察 Agent 读了什么、写了什么，以及如何组织记忆。
- **持续关联邮箱（smail）**：能持续获取新邮件、关联历史往来，结合个人资料拟好回复草稿；你可以编辑草稿后再发送，发送的应当是你确认的最终版本；重复检查邮箱不应重复处理或发送同一封邮件。
- **办理 ehall 事务**：能在 ehall.nju.edu.cn 中查询信息，并完成至少一种事务的材料准备和表单填写。行为要保守：提交前展示关键字段及后果，由你明确确认——退课、撤销申请等操作尤其不能自行决定，万一 Agent 把你的课都退了，你就完蛋了。
- **手机端联动**：能方便地从手机发起任务、查看进度、编辑草稿和确认操作；可以做 App、网页，也可以借助手机已有功能（例如在 Mac 上用 AppleScript 操纵备忘录再通过同步与手机联动）。任务不能依赖桌面端某个始终开着的对话窗口。
- **能力组合**：上述功能应当能共享必要的上下文，完成跨工具的任务；如何组合由你决定，但至少要做出一条你愿意反复使用的完整流程。

这五项能力看起来全部都在 AI 的能力范围之内，那为什么要布置这样的实验？讲师给出的理由是：如果你把实验要求直接丢给 AI，它一开始就会做出很多**看起来合理**的决定，但这些决定会让项目的维护变得越来越困难，最后直到陷入“焦油坑”（tar pit）。每个同学都有必要体验一下软件工程里的焦油坑——也就是当你的 AI slop 不断消耗 token 的时候，你终于有一天会下定决心：把这个项目删掉，从头重做，或者回退到某个很早期的版本。



图 1: 课程页面上 Lab 1《会成长的个人助手》的实验要求：五项基础能力逐条列出，并给出“收到办事通知 → 检索资料 → 手机追问 → 准备表单和草稿 → 归档”的示例流程[1]

1.1 软件项目的三层：intent、spec、code

回顾上一讲：项目是各种文件的集合。讲师特别强调他不喜欢 AI 写出来的那种非常长的 README。它应该写你在设计这个项目时的**最初初衷**：你想做一个什么样的产品、这个产品能达到什么效果，配两张截图，再加上最重要的那些设计。他举了一个例子：有一天你突然想到一个非常好的点子，想用它去挣 **1 亿**；要组建创业团队说服别人出来干，其实只需要一个比较短的 README——这就是你的 intent。剩下的部分，比如架构能力、写代码能力，合作者可以补出来：你不需要知道这个软件系统具体是怎样，它的 specification 是什么、用什么样的数据库、怎么支持用户从 1 到 10 到 100 到 100 万。

于是项目天然分成三层：**intent**（想解决什么）、**spec**（选什么数据库、什么编程语言、项目结构、每个模块的接口）、**code implementation**（实现本身）。它们都以文件的形式放在项目里。有了这样的项目，很自然就需要一个“容器”把这些文件管理起来——这就是版本管理的出发点。



图 2: 幻灯片“项目是各种文件的集合”：左侧白板画出 intent.md、spec.md、assistant.py、memory.md、test_assistant.py 等 artifact 及其关联（约定行为/实现/记忆/使用说明），右侧列出 Intent、Spec、Code 的分层[2]

下一讲预告：一个“冲突”的需求

讲师预告：下一个 Lab 会和这个 Lab 有非常大的关联，而且“你们暂时猜不到”。当你把 Lab 1 做成一个非常舒适的个人助手之后，他会“啪”地加一个需求——这个需求和前面所有需求都关联、并且有一定的冲突。你让 Agent 一针就去改，然后越改越崩溃。如果你现在选择躺倒、让 AI 直接做，下一次再把它删掉，就得不到这种软件工程体验了。

1.2 本章小结

本章回答的问题是“这门课的实验想教会什么”。Lab 1 用一个每个人都需要的需求（个人助手）作载体，真正要让你体验的是：**看似合理的一步步决定，如何在项目里积累成维护不下去的技术债**。理解了项目是 intent/spec/code 的集合，下一章就自然引出“这些文件之间的关系，文件系统到底装不装得下”。

[1] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:05:15–00:05:30。若为裁剪图，时间区间仍对应原视频画面。

2 文件系统的原罪：可追踪性与软件形态

文件系统管理项目是今天的默认做法，但它其实是历史的安排；这一章要回答的是：它为什么会成为默认做法，以及如果重来一次我们会怎么组织软件。讲师说：回到 1950 年、1960 年的时候，操作系统给你的存储功能就只有文件——那时关系型数据库还没有发明出来，所以所有程序员别无他法，只能把项目以文件的形式存进来。这是历史不停往前累积造成的结果，而他自己认为文件系统可能**不是组织项目的正确方式**，因为文件系统丢失了很多“文件不适合表达”的信息。

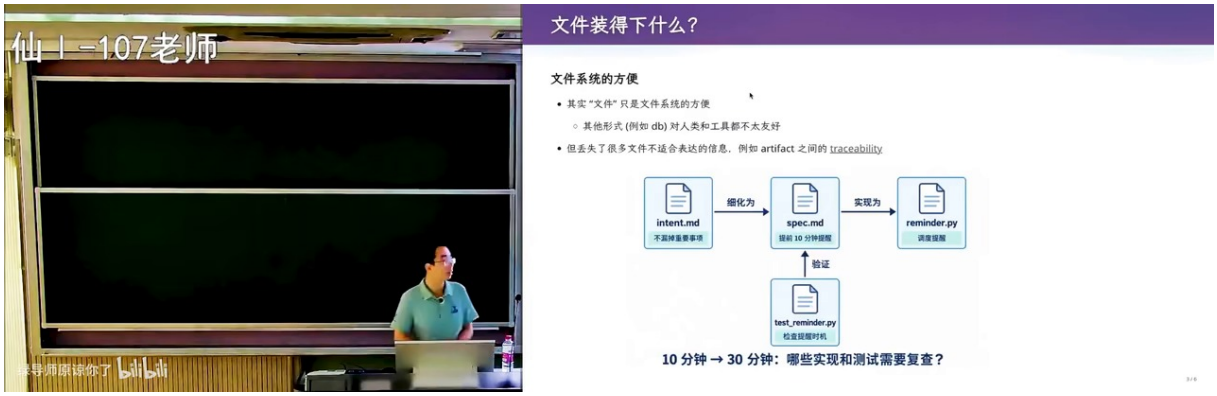


图 3: 幻灯片“文件装得下什么?”：“文件”只是文件系统的方便，但丢失了很多文件不适合表达的信息，例如 artifact 之间的 traceability；右侧白板画出 reminder.py、spec.md、test_reminder.py 之间的“细化为/实现为/验证”关联，以及“10 分钟改为 30 分钟：哪些实现和测试需要复查?”的问题[3]

2.1 技术债如何产生：一个 linked list 的例子

举一个最常见的例子。你让 AI 把链表（linked list）换成更高效的树（tree）结构去优化性能，AI “啪啪”就把代码改了。但你可能没注意到，某个 markdown 文档里写着“我这个项目的数据是用 linked list 管理的”。写文档的时候代码可能还不存在，所以你随手一写；一旦代码实现了，这个文档与代码之间的关联并没有被强行系上——这就是**技术债（technical debt）**。随着文档和代码不断向前推演，它们的不一致只会越来越多，不会越来越少。

更麻烦的是，当后面你（甚至 AI）拿出一份代码时，到底是文档说的对还是代码说的对？你跟 AI 聊天的历史没有了——你换了一台电脑，chat 已经消失；仓库里可能还留着一些 change 的日志，但你到最后真的不知道该信哪个。软件就此陷入维护的泥潭。

在软件工程里，这是一个被研究了非常长时间的问题，叫 **traceability（可追踪性）**：项目中的各种测试用例、文档、哪怕文档中的某一行和某段代码之间的关联是什么？我们能不能在软件系统的演进过程中仍然维持它？

[3]视频画面时间区间：00:06:45–00:07:00。若为裁剪图，时间区间仍对应原视频画面。

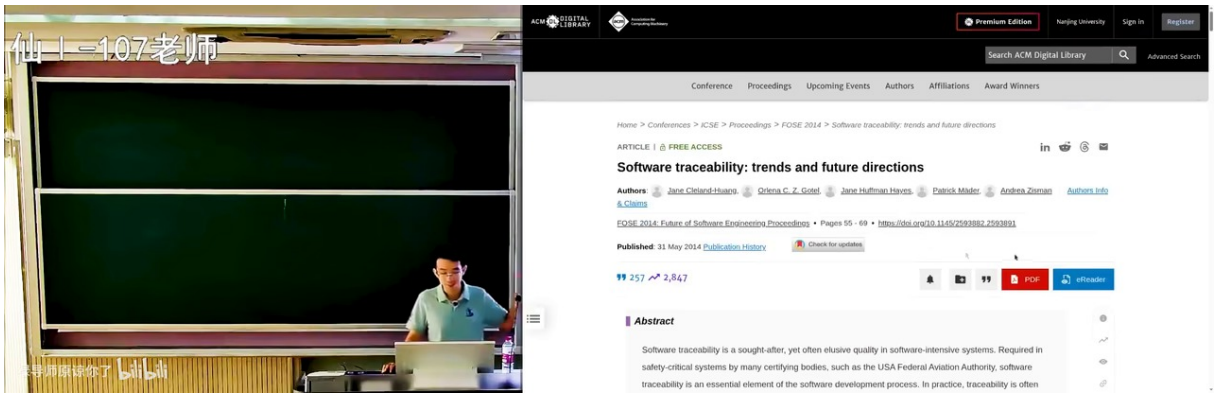


图 4: 讲师现场展示的 ACM Digital Library 论文页面：《Software traceability: trends and future directions》（Mader & Zisman，FOSE 2014，页码 55–69）[4]

2.2 如果重新设计：repo.db 与“用魔法打败魔法”

那么下一个是什么？从压缩包到 SVN，再到 Git，再下一个呢？讲师展示了一个他自己一直思考的推论：如果以后我们几乎不再直接管代码，那软件就不应该像现在这样在 Git 仓库里以一堆文件（README.md、hello.c、docs/ 下的文档）的形式存在。文档可能是好多个**切片**，把它们变成好多个对象（object / entity）：这个文档和这个实现之间“实现了”；一个测试用例“测试”这个文档，又“覆盖”那部分代码；文档之间又互相关联。这是一个完全结构化的东西，有环（cycle）的图，没有办法在文件系统里用树状形式表达。

这样的结构可以放到一个数据库里，就叫 `repo.db`。显然，人类没有办法用现在 Unix 世界里的工具去维护这样一个 repository 了。但不要紧——**用魔法打败魔法**：人类理解不了的东西，我们只要训练 AI，它只要有强化学习的反馈信号，AI 就能够理解这个 repo。

古法编程的下一步

用文件系统管理代码，你总有一天受不了了：打压缩包发布、打压缩包发给合作者，这件事已经维护不下去了，所以有了 SVN，有了 Git。而“下一个是什么”，正是这一讲要讨论的问题——先在 Git 的模型里找它的缺陷，再在 Agent 时代重新设计。

这个设想落到日常工作上是这样：每次你只做一小步（fix 一个 bug、改一个文档、对幻灯片样式不满意），大型系统里绝大部分内容跟这一小步无关。你可以让 AI 根据你的 chat 把相关的东西摘出来——比如这个文件、这个 test case 和这个文档——其他都不需要。AI 有动态生成任何内容的能力（**everything is code**，content as code），所以它可以根据 chat 生成一个与这一小步相关的**视图**（数据库里 View 的概念：数据就在那儿，但可以创建一个不一样的查询，以另一种形式展示出来）。改完之后，它仍然是从一个一致的 repo.db 到下一个版本的 repo.db：一次小步修改，把错综复杂的关系推进到下一个一致状态。从此软件工程“彻底变了”——在 AI 内部，软件是这样生产的，而我们也就再也不要看代码了。

[4] 视频画面时间区间：00:09:45–00:10:00。若为裁剪图，时间区间仍对应原视频画面。

熟悉的概念会用上：数据库的实体、关系与视图

这里借用了数据库课的三个概念：实体（entity）、关系（relation）、视图（view）。讲师强调，创造这些新东西的过程里，用的都是经典计算机科学课程的知识——数据库的实体与关系、Git 的“小步向前”模型。如果在这个模型底下我们是大步向前的，那就惨了。也可能模型变得非常强、一个大型项目一步就变成下一个，但至少在初期，我们还是要沿用过去积累的软件工程智慧小步前进。

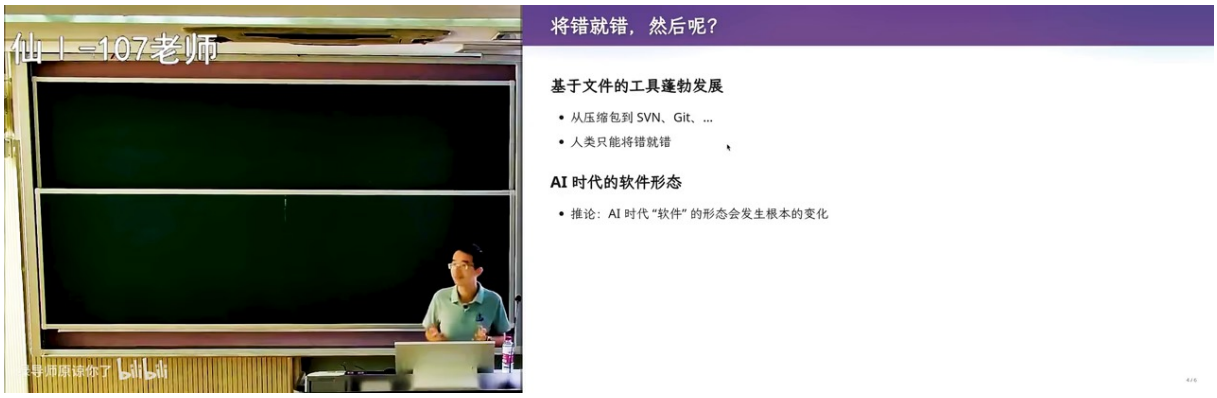


图 5: 幻灯片“将错就错，然后呢?”：基于文件的工具蓬勃发展（从压缩包到 SVN、Git），人类只能将错就错；推论是 AI 时代“软件”的形态会发生根本的变化[5]

2.3 本章小结

本章回答“为什么说文件系统是原罪”。文件系统是 1950 年代历史条件的产物，它丢失了 artifact 之间的 traceability，技术债因此只增不减；学术界为此研究了数十年。重新设计的方向是把软件表达为结构化的实体-关系图（repo.db），让 AI 借助反馈信号去理解和维护它，用视图把“与这一小步相关的东西”呈现给人。下一章回到 Git 本身，看看它的模型到底哪里不够好。

3 Git 的模型：快照、merge 与 rebase

在讲重新设计之前，先复习 Git 到底管理了什么。Git 管理的是**文件、目录、快照和快照之间的关系**：你每做一件事情——fix 一个 bug、改一个文档——都产生一个 commit；这样多个人之间可以协作，每个人产生自己的 commit，再用 rebase、merge、fast-forward 这些操作创建一个可追踪的整个项目的修改历史。

[5] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。



图 6: 幻灯片 “Git 管理了什么?”：文件、目录、快照和快照之间的关系（A +feat:... → B，Merge A,B→C）；并行开发与冲突；Merge 汇合历史，rebase 把修改依次重放到新的起点[6]

3.1 merge 的隐藏问题：作者归属被改写

如果你要把两个开发者的工作合起来，在 Git 的原始模型里更提倡用**两路 merge**：创建一个有两个 parent 的提交，继承双方的修改。但这又带来一些问题。比如 merge 是由 A 来干的，那 merge 里面由 B 写的那些内容，作者就变成 A 了——存在一个隐藏的作者修改。再比如发生冲突、由 A 来 resolve 的时候，冲突解决完之后这个 change 就算 A 完成的，B 必改的代码最后 ownership 归到了 A。这会带来一些小麻烦：出了问题去追溯，看到的名字可能并不是真正写那段代码的人。

3.2 rebase：把修改重放到新的起点

rebase 解决的是另一件事。假设 A 同学先改了文档、又加了测试（产生两个 commit，德尔塔一、德尔塔二），B 同学加了一个 feature、又加了一个测试。rebase 不是直接把两条线路并起来的提交，而是试图先**把 B 的修改重放到 A 的新起点上**：能不能在 A 的后面把这个 feature 加上？能加上就成功了，再把 test 搬过来；如果两条线上的修改完全没有冲突，就可以一路合并成线性的历史。

但这里有很大的风险：B 加的测试用例可能和 A 的 feature 不兼容，甚至两边改了同一个地方、做了两个完全不兼容的东西。一旦发生冲突就需要 resolve，你会进入 rebase 模式——进过这个模式的人都知道，瞬间自己就不知道应该怎么办了，然后打开 Agent 问 “我现在到底发生了什么”。更麻烦的是，就算 feature 搬过来了，搬 test 的时候又可能失败；而且**每重放一次就创建一个新的 commit**：rebase 的本意就是把 base 改掉，base 一改，新的 commit ID 就产生了，历史也随之改变。

3.3 我们真正想操作的是什么？

这里能感觉到一点点轻微的 “不和谐”：我们关心的是**修改**本身—— “把这个修改搬过来” “把那个修改搬过来”。但在 Git 里，修改是用 commit 来替代的：一个 commit 和它的 parent 之间的关系就叫一个修改，没有一个专门的叫 change 的对象。绝大部分情况这都能满足，但如果把 change 也变成 first-class citizen，会发生什么呢?

[6] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

同学产生的疑问：学生时代用不到 rebase？

有同学（已毕业）提了一个 valid point：学生时代大多数人不会和他人共同合作，都是单分支开发，rebase、merge 用的场景就很少。讲师的回应是：往外面的世界走一走，互联网开发者社区铺天盖地都是文档，你一定会看到仓库根目录下的 `CONTRIBUTING.md`——一份贡献者指南，通常回答“我怎么参与这个项目”，里面有项目总体结构、分支与提交信息、PR 的规范和流程。看到“必须在自己分支上 rebase、保持线性历史、禁止直接创建分叉 merge”这样的规定，你就知道 rebase 是干什么的了。



图 7: 幻灯片“同学产生的疑问”：左侧是同学关于 git rebase 的原始评论，右侧是讲师的回复与“典型的学生思维”（老师不教的就是我不需要会的、考试不考的就是我不用学的）[7]

这背后还有一层“学生思维”的讨论：高中时老师不教的就是你不需要会的，考试不考的就是你不用学的——为了升学的利益最大化，这是正确的；但到了大学还带着这个思维，就会错过很多东西。学习的目的，是形成自己的理解（first principles）：AI 时代的“斩杀线”以下的价值会归零，低级的智力劳动（比如大量会计从业者——报道说中国可能有一**千万**会计从业者）正面临被 Agent 替代的危险，所以更要在这门课里学到经典的东西，最终形成自己的理解。



图 8: 幻灯片“如果想重新设计版本管理系统?”：Git 经常需要处理 commit 之间的 delta（cherry-pick、rebase……），但这是用 commit 来替代的；我们究竟想操作什么——“把这次修改搬到另一个版本上”，我们关心的是 change[8]

[7] 视频画面时间区间：00:19:15–00:19:30。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:22:45–00:23:00。若为裁剪图，时间区间仍对应原视频画面。

3.4 本章小结

本章回答“Git 的模型哪里不够好”。Git 管理快照和快照关系，用 commit 代替 change：merge 会改写作者归属，rebase 每次重放都会产生新的 commit ID 并带来冲突风险。我们真正想操作的“同一次修改”，在换了一个起点之后，在 Git 的模型里失去了身份。下一章沿这条线索推下去，就得到了 Jujutsu。

4 把 Change 作为一等公民：Jujutsu 的设计

沿着上一章的追问——如果 change 也变成 first-class citizen，会发生什么？——你就发明了下一代更好用的版本控制系统：**Jujutsu (jj)**。它和 Git 底层完全兼容，你可以直接把它当 Git 用；但它增加了一个叫 change 的概念，每个这样的东西都可以被赋予一个 change ID。

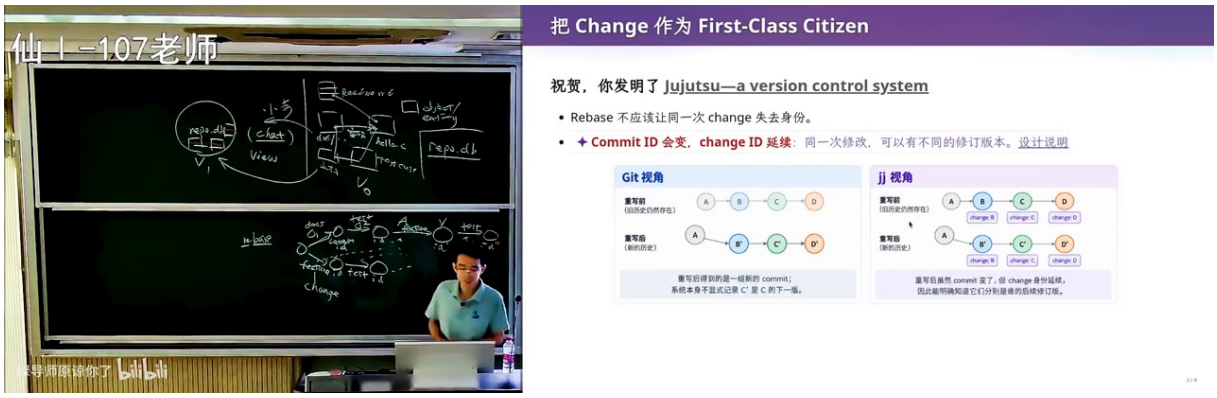


图 9: 幻灯片“把 Change 作为 First-Class Citizen”：“祝贺，你发明了 Jujutsu——a version control system”；Rebase 不应该让同一次 change 失去身份；下方对比 Git 视角（重写后得到的是一组新的 commit，系统不显式记录 C’ 是 C 的下一版）与 jj 视角（commit 变了，但 change 身份延续）[9]

4.1 change ID 延续，commit ID 会变

核心区别在 rebase 时的行为。下表把三种操作放在同一张表里对比（三行都是讲者在课上逐条讲的）：

操作	提交身份	冲突处理
Git 两路 merge	创建一个有两个 parent 的新提交，原有 commit ID 都保留；但 B 写的内容作者会变成执行 merge 的 A	冲突在 resolve 时解决，解决后的 change 算 A 完成的
Git rebase	每重放一次创建一个新的 commit ID，原 ID 作废；系统不显式记录 C’ 是 C 的下一版	一旦冲突就进入 rebase 模式，逐个 resolve；搬 feature 和搬 test 都可能失败
jj rebase	Commit ID 会变，但 change ID 延续：同一次修改可以有不同的修订版本	冲突的逻辑表示直接存进 commit，带着冲突仍能继续 rebase、merge，之后再统一处理

表 1: Git 与 Jujutsu 在 rebase 上的对比（据幻灯片 fig_09/fig_10）

[9] 视频画面时间区间：00:28:45–00:29:00。若为裁剪图，时间区间仍对应原视频画面。

在 Git 里，rebase 每重放一次就创建一个新的 commit，原来的 commit ID 作废；系统本身不显式记录 “C’ 是 C 的下一版”。而在 jj 里，rebase 是把 **change 直接搬过来**：Commit ID 会变，但 change ID 延续——同一次修改，可以有不同的修订版本。即使冲突解决不了，它也可以把 change 带下来生成一个直接 rebase 到目标位置的提交，里面引入的冲突用 Git 的冲突记录，但 jj 有一套自己的记录方式，把所有 change 的冲突也塞进去；然后你在那里 resolve，底层的模型没有变，但用起来会舒服很多。

4.2 连 staging area 都不需要了

jj 还有一个很好的特点：不需要 staging area。想想在 Git 里最讨厌的场景：你有一个工作区，改 hello.c 改到一半，项目还不能编译、不能运行、测试也通不过，突然导师打电话说有一个高优先级的 bug 要一分钟修。你不应该 commit，所以需要把改动加到 index 里，然后 `git stash` 压栈。更糟的是，刚修了一半，导师又打电话说这个 bug 别修了，还有更优先的——你又得再 stash 一次。stash 是一个栈，里面会堆好多层，你还得去管理这个栈。



图 10: 幻灯片“让未完成的工作也有位置”：连 staging area 都不需要了，仿佛一直在 commit+amend；工作副本本身就是一个 commit；冲突直接存进 commit（killer feature）；保存的是冲突的逻辑表示，带着冲突的 commit 仍能继续 rebase、merge[10]

在 jj 的模型里，如果你想象自己随时随地都在编辑这个 repo——改 hello.c 加一行减一行，都是一个 change。它好像是你但凡改了一个字节，就立即做了一个 commit；当然这不会产生线性的历史，而是先把上一次的 commit 抹掉，用当前修改去替代它——也就是永远处于 “一个 change 提交的状态”（仿佛一直在 commit+amend）。于是 staging area 也不需要了，未完成的工作天然有了位置。

设计启发：把“尚未解决”表达成可保存的状态

幻灯片上的原话是：保存的是冲突的**逻辑表示**，带着冲突的 commit 仍能继续 rebase、merge。把 “尚未解决” 表达成一种可保存、可继续操作的状态——这是一个可以推广到很多系统设计里的想法。

[10] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

4.3 肌肉记忆与学习的方向

那么为什么大家还在用 Git？讲师的答案是**肌肉记忆**：他已经形成了 Git 的肌肉记忆，比他更老的一代形成了 SVN 的肌肉记忆；人一旦形成肌肉记忆，把习惯掰过来就很困难。即便他能从 first principle 理解 Jujutsu、甚至 virtual reimplement 这个系统，也没有足够的时间再去训练另一套肌肉记忆。

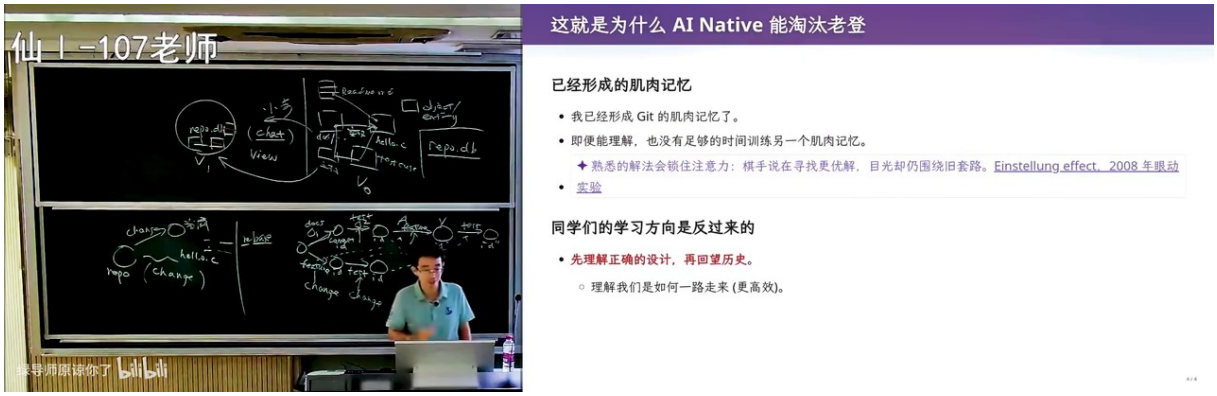


图 11: 幻灯片“这就是为什么 AI Native 能淘汰老登”：已经形成的肌肉记忆；熟悉的解法会锁住注意力（Einstellung effect，2008 年眼动实验）；同学们的学习方向是反过来的——先理解正确的设计，再回望历史[11]

幻灯片上引用了一个认知科学的发现：熟悉的解法会锁住注意力——棋手自以为在寻找更优解，目光却仍围绕旧套路，这叫 Einstellung effect（2008 年的眼动实验）。所以同学们的学习方向是**反过来的**：你们现在还没有形成记忆，就应该直接去用那个“对的设计”（先用肌肉记忆记住最正确的设计），然后再回望历史——“原来 Git 这么蠢”“原来 SVN 从 diff 那条线一开始就走错了”。Git 出现时大家觉得它先进，因为它是 persistent data structure；到今天回头看，Git 也犯了一些设计上的错误：它最早的时候没有想过 rebase 会有这么大的影响，有 merge 就已经能解决手上的问题，所以没有把概念模型想得更清楚。

4.4 本章小结

本章回答“change 作为一等公民能换来什么”。Jujutsu 让 change ID 延续、commit ID 可变，rebase 不再让同一次修改失去身份；工作副本本身就是一个 commit，staging area 与 stash 都不再必要；冲突可以被保存为逻辑表示并继续操作。它对人类更友好，也可能对 Agent 更友好。学习方法上的启示是：先学正确的设计，再回望历史。下一章进入本课的正题——为 Agent 重新设计版本管理。

5 为 Agent 重新设计：从单线程人类到 Agent 团队

这门课不是软件工程，而是生成式软件工程。所以真正要做的事是：在 Agent 时代**再次重新设计版本管理系统**。你可以让 Agent 去用 Git，也可以让 Agent 去用 Jujutsu；jj 对人类是一个很大的进步，但对 Agent 来说进步可能相对小一些——因为它好像很会 rebase。我们一路走过来，SVN 有局限，Git 有局限；那在 Agent 时代，我们的局限和要破掉的改进是什么？

[11] 视频画面时间区间：00:33:45–00:34:00。若为裁剪图，时间区间仍对应原视频画面。

5.1 所有 VCS 的共同假设：使用者是人类

刚才所有的 version control system 都有一个假设：使用者是人类。人类有什么特点？人类是 **single-thread** 的生物。讲师用操作系统的并发来打比方：把线程想象成一个人，线程访问共享内存，两个基本操作是 load 和 store（或者说 read 和 write），这两个操作不能同时进行——既读又写就需要原子指令。大部分共享内存访问就像人一样：想观察世界时手要抱在后面，只能看；等看完了想施加效果时，必须把眼睛闭起来——而眼睛闭起来的时候，你上次读到的那个值其实已经失效了，可能经历了很多事情（被换 CPU、被中断打断、别的线程上来执行了一秒），你再把“粉笔”扔出去，很可能就扔错了。

人类之间还有无法逾越的物理屏障：你不能理解我的思想，我也不能理解你的思想。我们只有一份 I/O 设备——一双手、一张嘴、一对眼睛。以这种方式编辑代码库产生 change，是以**小时**为单位的。一个软件公司负责一个模块的开发可能多也就十几个人，十几个人每人以小时为单位开发，最多也就是以**十分钟**为单位产生一个提交。



图 12: 幻灯片“Agent 时代，再次重新设计版本管理系统”：人类是 Single-Threaded 生物，只有一份 I/O 设备；Git 基本够一个人用了——stash 把手头工作放一边、branch 保留不同方向的尝试，Jujutsu 就更好用了[12]

十分钟一个提交意味着什么？意味着提交速度会比人和人之间**沟通**的速度慢很多。你写一个东西要一两个小时，写之前肯定先跟师兄说一声“我要干个这个行不行”，二十秒三十秒沟通完就去改；或者去厂里打工，每天早上的例会把任务分享出来，提前在半小时里划分 scope，把项目切开，领一部分走，再去产生 change——这是古法编程。在古法编程时代，stash 是一个合理的设计：导师给你打电话是可能发生的，但你修到一半时导师再打电话，概率就很低，所以绝大部分时候 stash 栈里就一层。而你会尽可能保证 stash 不要太深，因为你可以拒绝一个工作（“我手上有一个更高优先级的工作”）。

5.2 Agent 提速之后，Git 跟不上

但在 Agent 时代，这件事变了。如果是 agent team（swarm），单个 agent 生产 AI slop 的速度就远远快于人：它生成 AI slop 以**分钟**级为单位，甚至更快——你让它 fix 一个东西，上下文是热的时候可能就几十秒，DeepSeek flash 干活几秒就做了。Agent 工作时以**秒**级可以产生一个原子提交，并且你可以轻易得到 **10 倍、100 倍**的 agent team——Git 或 Jujutsu 这样为人设计的模型就有点跟不上了。

[12] 视频画面时间区间：00:35:53–00:36:08。若为裁剪图，时间区间仍对应原视频画面。

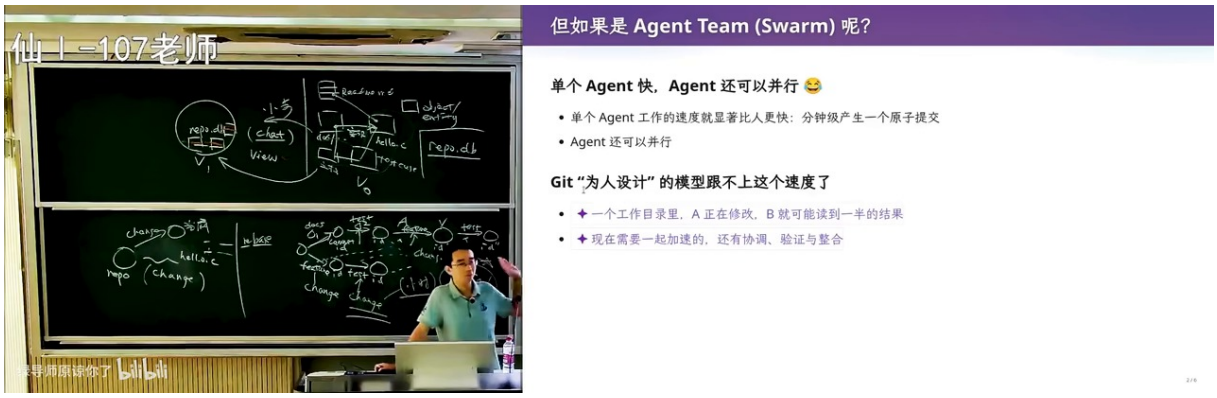


图 13: 幻灯片 "但如果是 Agent Team（Swarm）呢?"：单个 Agent 分钟级产生原子提交、Agent 还可以并行；Git 为人设计的模型跟不上这个速度——一个工作目录里 A 正在修改，B 就可能读到一半的结果；需要一起加速的还有协调、验证与整合[13]

就算是大型项目有一些门控（比如写完必须经过几分钟的测试流水线）可以放缓 Agent 提交的速度，毫无疑问，Agent 加速了软件开发的过程。那我们应该怎样为 Agent 设计版本管理系统?

5.3 Workaround 一：每个 Agent 一台开发机

首先可以让 Agent 沿用人的模型：每个 Agent 你就想象它有一个自己的开发计算机——严格做 human:agent 的一一对应类比。给它一个独立的电脑，可能是个容器，放到某个机器上，然后直接 instruct Agent：你的 remote 仓库在某处，你现在就是一个单人开发者，可以从 tmux 里收到命令，收到命令就可以开干。这非常容易实现。讲师现场演示：一个 master 加两个 tmux session（实际开发时这两个 session 位于两台不同的机器或容器，物理上隔离，看不到世界上其他人）。Agent 时代的好处是**每个人都是领导**——你就是包工头。

他还演示了一个流行的 paradigm：你有一个监工的 agent，有好多个子 agent；为了最大化模型的智力、防止 AI 生成你不想要的东西，放两个比较强的模型，比如由 Codex 来设计、让 Claude 来 review，让它们互相打架达成一致，主 agent 负责协调、观看输出或总结报告，往里面 inject prompt，多 agent 系统就转起来了。如果大家工作在同一个 Git 仓库上就会有一些麻烦，但工作在不同的 repo 上就完全可以：可以拉取别人的提交，冲突了就自己修复，必要时还能给 master agent 发消息。

5.4 Workaround 二：git worktree

除了多机多容器，还有今天的 work around：Git 自带 **worktree** 功能。Git 允许我们创建一个工作树，你可以想象它连接到了一个 remote；可以在隔离的环境里做独立的提交。大家都说 `.git` 是一个目录，里面有 logs、objects 等三种类型的对象；那 worktree 里面的 `.git` 是什么呢? 它是一个**普通的文本文件**。

[13] 视频画面时间区间：00:40:53–00:41:08。若为裁剪图，时间区间仍对应原视频画面。

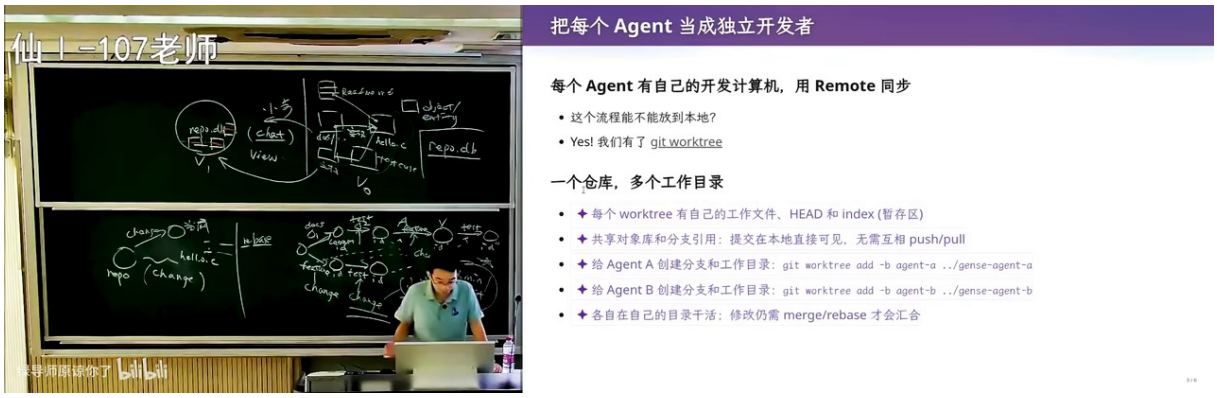


图 14: 幻灯片“把每个 Agent 当成独立开发者”：每个 Agent 有自己的开发计算机，用 remote 同步；一个仓库多个工作目录——每个 worktree 有自己的工作文件、HEAD 和 index（暂存区），共享对象库和分支引用，提交在本地直接可见，无需互相 push/pull；给 Agent A/B 创建分支和工作目录用 `git worktree add -b agent-a ../gense-agent-a`[14]

```
git worktree add -b agent-a ../gense-agent-a
git worktree add -b agent-b ../gense-agent-b
# 各自在自己的目录干活；修改仍需 merge/rebase 才会汇合
```

Listing 1: 幻灯片上给出的 worktree 用法：一个仓库，多个工作目录

如果你继续追问 Agent，它会告诉你：`.git` 除了可以是目录，还可以是一个只有一行格式‘gitdir: ...‘ 的文件，指向主仓库的`.git/worktrees/`。所以 Git 最大程度上复用了操作系统里的机制：每个 worktree 是一个微型的 Git 目录，依然有 HEAD、index、logs，但依附于原有的 Git 目录而生。这个特性在 AI 时代突然变得有用：多个 Agent 的提交在本地直接互相可见，不需要互相 pull 和 push；它提供了比多台虚拟机更轻量的合作方式。但有冲突的修改仍然需要 merge 和 rebase 才能合并。

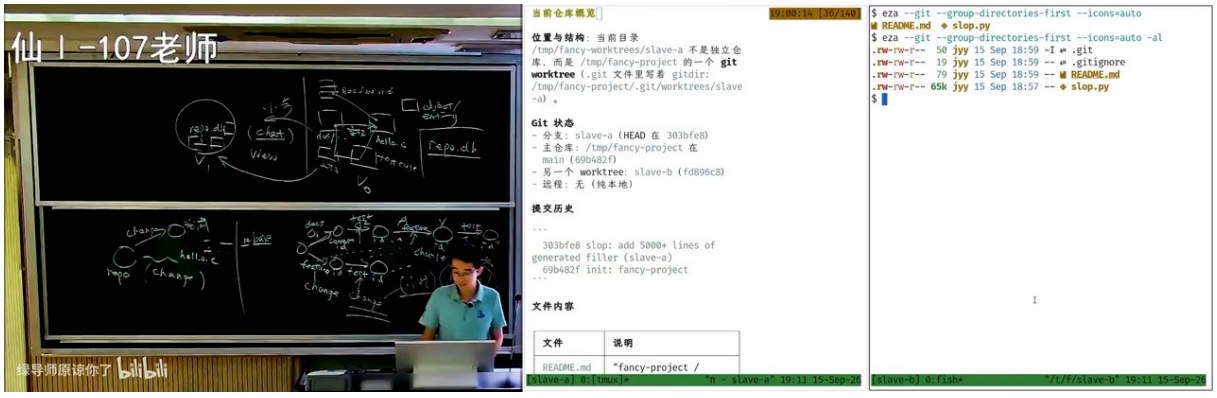


图 15: 讲师现场演示：worktree 里的`.git` 是一个文本文件（内容为 gitdir: /tmp/fancy-project/.git/worktrees/slave-a），Agent 给出的“当前仓库概览”说明了位置与结构、Git 状态（分支 slave-a、主仓库 main、另一个 worktree slave-b）与提交历史[15]

[14]视频画面时间区间：00:42:08–00:42:23。若为裁剪图，时间区间仍对应原视频画面。

[15]视频画面时间区间：00:49:08–00:49:23。若为裁剪图，时间区间仍对应原视频画面。

5.5 本章小结

本章回答 “Agent 时代要破掉什么假设”。人类是 single-threaded、以小时为单位产生 change 的生物，十分钟一个提交的节奏与沟通成本塑造了 stash、branch、每日例会这些实践；Agent 以秒级产生原子提交、还能 10 倍 100 倍地并行，为人设计的模型开始跟不上。两个立即可用的 workaround 是：每个 Agent 一台开发机加 remote 协调，以及 git worktree（一个仓库多个工作目录，本地可见、无需 push/pull）。下一章进一步追问：并发控制的方式要不要换。

6 并发控制的新方向：从乐观到悲观

有了 agent team 之后，除了 worktree 这样的 work around，还要回头想**为什么我们需要版本控制**：人的限制塑造了工具。人的生产效率有限；人之间有 communication gap，实际上沟通效率极低；而且人就算沟通了，也可能各种出错——你的室友向你请教一个数学问题，明明你会做，却怎么都讲不明白，除非你是很好的老师、能共情别人。这也是大厂和小厂的效率差异：软件系统大了、人多了，沟通效率变低，产出速度就降下来；一个 **100** 人的团队不可能达到 10 人团队效率的 10 倍，能有 2、3 倍就不错了。

所以我们需要 Git 的 best practice：快照、小步前进（改一个 typo 也要 commit）、每个 commit 都可追溯。一旦出问题，需要回溯到那个人做 commit 的时间点——那个人是有记忆的，他会回想起那天早上的会、那段沟通记录，然后说 “我们这个地方应该怎么修正”。这就是 `git bisect`、`git blame` 这些专门为人设计的特性。

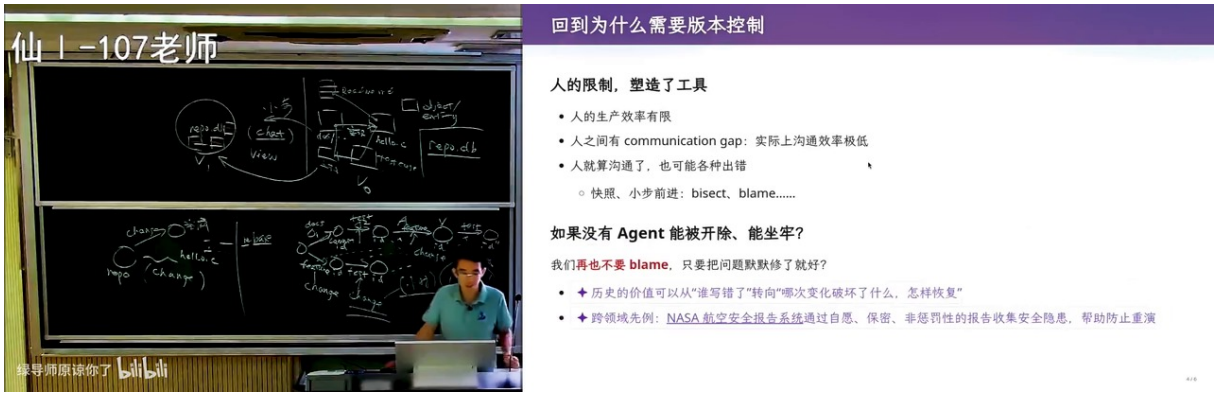


图 16: 幻灯片 “回到为什么需要版本控制”：人的限制塑造了工具——生产效率有限、communication gap、沟通了也可能出错；快照、小步前进，bisect、blame；如果没有 Agent 能被开除、能坐牢? ——历史的价值可以从 “谁写错了” 转向 “哪次变化破坏了什么，怎样恢复”；跨领域先例是 NASA 航空安全报告系统[16]

6.1 我们还需要 git blame 吗？

想一想：如果由 agent team 或 agent swarm 来实现你的项目，我们还需要 `git blame` 吗? blame 的前提是人可以被问责——这个事情你多了就该被开掉，实习生就背锅了。但 Agent 没有被开除、被坐牢的能力，这个模型就变了：我们再也不需要 blame，项目只要前进就可以了。有时甚至不需要知道是哪个提交引入的错误——只要知道这个地方错了，把这个错误修了就行。

[16] 视频画面时间区间：00:51:08–00:51:23。若为裁剪图，时间区间仍对应原视频画面。

更好玩的是，每个 Agent 都是**相同模型的克隆体**：同样的事情让它们来做，虽然模型还有不确定性，但大概率会收敛到相同的结果。所以历史的价值可以从“谁写错了”变成“哪次变化破坏了什么、怎样恢复”——只要保存，我只要能知道从现在开始怎样往前进。幻灯片还给了一个跨领域先例：NASA 的航空安全报告系统通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演。快照仍然需要：这个方向走错了，如果没有保留历史上的某个 snapshot，一下子往前走了好多再回去，代价可能比一点一点往前修要大。

6.2 Git 其实是乐观并发控制

再想一步：Git 的模型更像是数据库里的**乐观并发控制**（optimistic concurrency control）。两条 SQL 各自是一个 transaction，中间可以混任何代码；先 select 读一下，然后做图灵完备的代码计算（Python 或 JavaScript，你永远不知道它根据读到的结果算出什么），最后 update——update 时就会产生冲突，甚至死锁（循环依赖），导致 transaction abort 回滚。

说它是乐观的，因为它假设两个 transaction 不会冲突。这个假设在数据库系统里大概率成立：想想教务系统，每个同学读写都是自己那块数据——选课选的是我的课，查课表查的是我的课表，两个同学同时开始选课，也是读我的写我的。Git 也一样：事先开过会、划分了 scope，两个人顺着各自的路先做，合并时大概率不产生冲突，merge 或 rebase 都能顺利完成。

6.3 Agent 更快之后：换用悲观并发控制

但如果 Agent 快了 **1000** 倍呢？主 Agent 一下子分出好多任务，两个 transaction 就真的会**更频繁地触发 abort**。在 AI 时代 abort 也不是大问题——Agent 出错或冲突了它就修。但我们是不是可以换一种并发控制的方式？

在操作系统里，你们是用什么做并发控制的？**上锁**。锁和数据库的 transaction 不一样：想要访问可能冲突的资源时先 acquire 一把锁，别人再想 lock 就必须等你 release。讲师上课常说的“一把大锁保平安”就是这个道理。也就是说，必须在使用资源之前 declare 对资源的使用——先声明，再动手，而不是“先往前跑，跑了再说”。

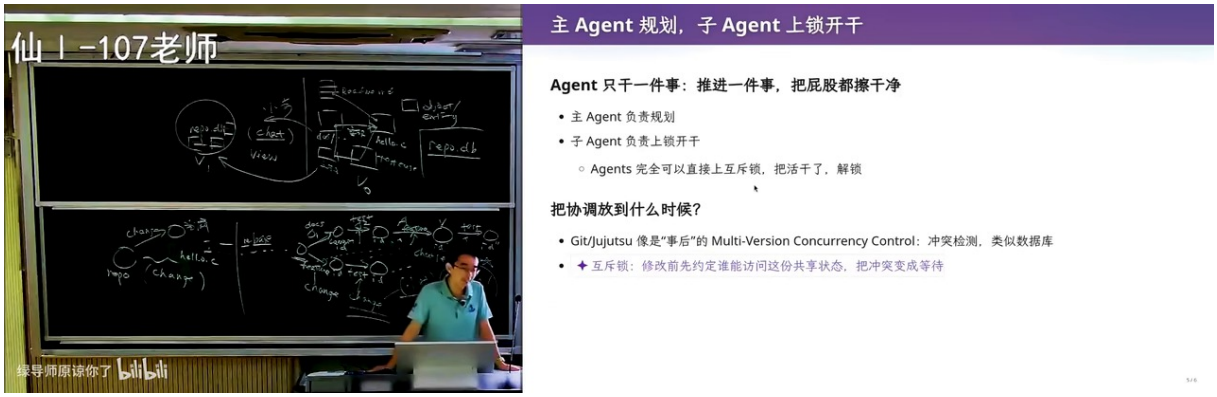


图 17: 幻灯片“主 Agent 规划，子 Agent 上锁开干”：主 Agent 负责规划，子 Agent 负责上锁开干；Agent 只干一件事、把屁股擦干净；Git/Jujutsu 像是“事后”的 MVCC——冲突检测类似数据库；互斥锁则是修改前先约定谁能访问这份共享状态，把冲突变成等待[17]

落到实践：Agent 的动作非常快时，我们应该用更加**悲观**的并发控制。主 Agent 规划完，立马把项目的某一部分锁上；在 Git 仓库或快照上支持一个锁的操作，把要用的 scope 锁住，做完 commit

[17] 视频画面时间区间：00:55:23–00:55:38。若为裁剪图，时间区间仍对应原视频画面。

后释放；另一边如果想要干的事情被别人锁上了，就必须等前面的人释放才能上锁——这就几乎避免了所有冲突。比如在文件集上锁，就避免了两个 Agent 同时写同一个测试或同一个配置。又因为 Agent 数量足够多、速度足够快，即使有 Agent 在等锁，整体进度仍然可以往前推。

互斥锁的本质是串行化

互斥锁的本质是**不要并行、不要并发**——它和多处理器并行是有根本矛盾的，它要把一个能并发的程序退回成串行程序。但当系统里有很多独立的、不冲突的线程在运行时，即便每个线程都持有一把锁，整体仍然可以是高效的。悲观不等于慢。

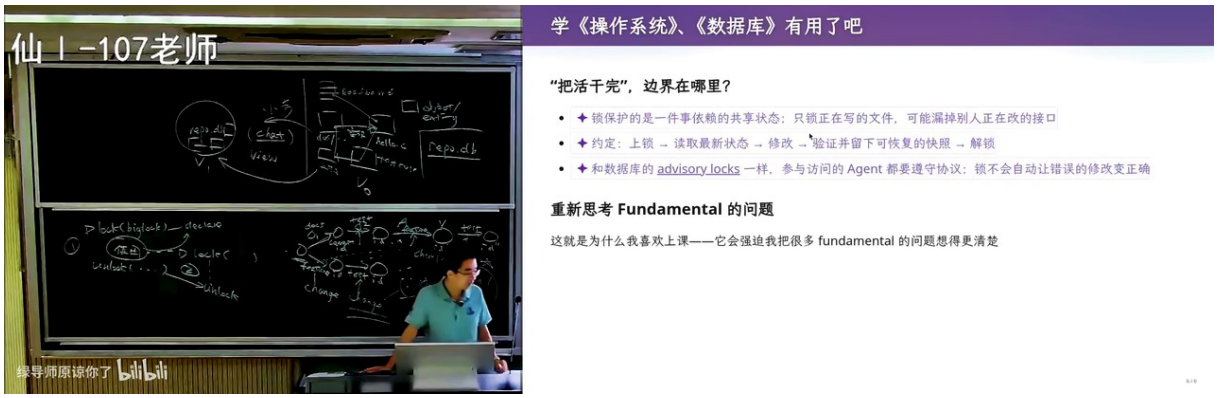


图 18: 幻灯片“学《操作系统》《数据库》有用了吧”：“把活干完”的边界在哪里——锁保护的是一件事依赖的共享状态，只锁正在写的文件可能漏掉别人正在改的接口；约定是上锁 → 读取最新状态 → 修改 → 验证并留下可恢复的快照 → 解锁；和数据库的 advisory locks 一样，参与访问的 Agent 都要遵守协议[18]

幻灯片还给出了“把活干完”的边界：锁保护的是一件事依赖的共享状态，只锁正在写的文件可能漏掉别人正在改的接口；推荐的工作流约定是**上锁 → 读取最新状态 → 修改 → 验证并留下可恢复的快照 → 解锁**。还有一条重要的提醒：和数据库的 advisory locks 一样，参与访问的 Agent 都要遵守协议——锁不会自动让错误的修改变正确。讲师说，这就是他喜欢上课的原因：备课会强迫他把很多 fundamental 的问题想得更清楚。

6.4 本章小结

本章回答“并发控制要不要换”。Git 的模型近似数据库的乐观并发控制，前提是 transaction 不会冲突；Agent 快了 1000 倍之后，冲突会频繁触发，而 Agent 时代连 blame 都不需要了——历史的价值从“谁写错了”转向“怎样恢复”。更适配的方案是操作系统的悲观锁：先声明、再修改、验证、解锁，主 Agent 规划、子 Agent 上锁开干。同时要记住：锁不会自动让错误的修改变正确。下一章从工具回到真正困难的团队管理。

7 工具之上：团队管理与“斩杀线”

版本控制本质上还是一个工具——不管是 Git、Jujutsu，还是刚才那种基于乐观或悲观锁的版本控制，真正困难的是**进度（团队）管理**。

[18] 视频画面时间区间：01:02:38–01:02:53。若为裁剪图，时间区间仍对应原视频画面。

7.1 AI 是一条“斩杀线”

我们为什么现在觉得还需要软件工程？因为我们没有办法做到一句话就直出一个高质量的产品。老板有一个想法——“我要做一个 Agent 用的微信”，随口说一说；如果这时候 AI 直接就能出那个微信级的产品，那软件工程就已经死了。但现在的斩杀线暂时还没有到团队管理这一层，所以我们还要学。

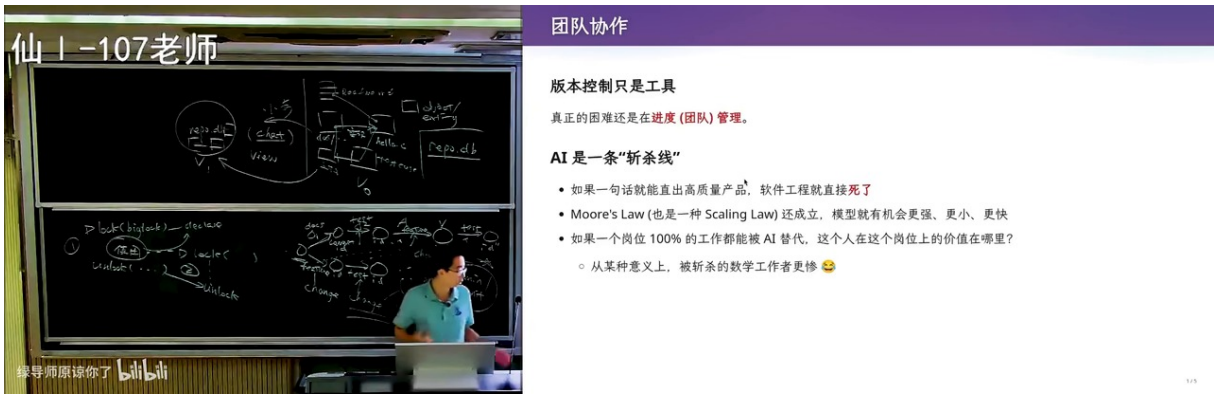


图 19: 幻灯片“团队协作”：版本控制只是工具，真正的困难在进度（团队）管理；AI 是一条“斩杀线”——如果一句话就能直出高质量产品，软件工程就直接死了；Moore’s Law（也是一种 Scaling Law）还成立，模型就有机会更强、更小、更快[19]

讲师也坦率地承认自己可能有一天也会被“斩掉”：因为摩尔定律本质上也是一种 scaling law，它现在还成立，模型肯定会更强、更小、更快，也许算力有一天能唤出我们的智能。但如果一个岗位 **100%** 的工作都能被 AI 替代，这个人在这个岗位上的价值在哪里？他半开玩笑地说，从某种意义上被斩杀的数学工作者更惨——AI 能把数学题做出来了，但数学系还在教证明。

7.2 团队的复杂性：平方级的沟通关系

至少今天 AI 还不知道怎么做团队管理。因为每多一个人，沟通关系都是平方级上升的。这是一个可以在笔记里写下来的数量关系：

$$C(n,2) = \frac{n(n-1)}{2}$$

其中 $C(n,2)$ 是 n 个人两两之间的潜在沟通关系数，n 是团队人数。$n=5$ 时是 **10** 条，$n=10$ 时是 **45** 条。这正是 Brooks《人月神话》里的教训：人月不互换，加人不提速。

[19] 视频画面时间区间：01:04:48–01:05:03。若为裁剪图，时间区间仍对应原视频画面。

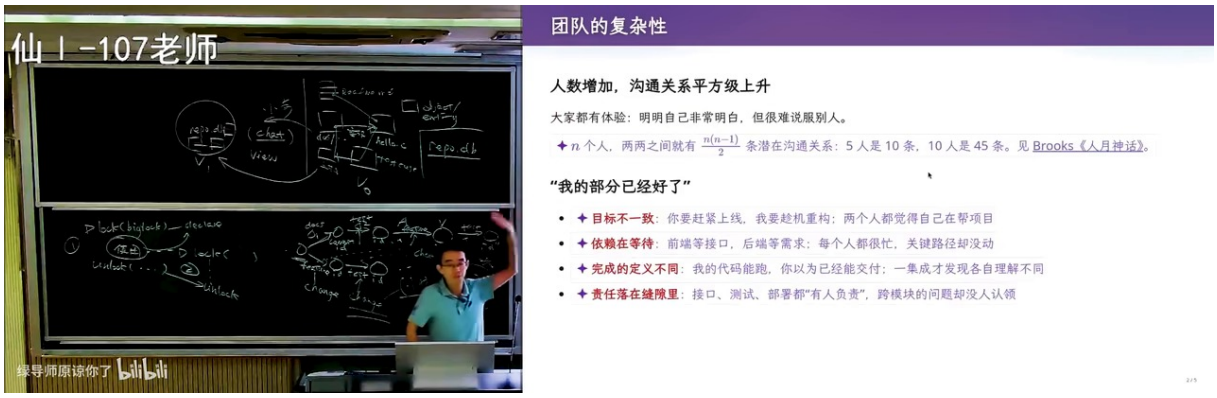


图 20: 幻灯片“团队的复杂性”：n 个人两两之间就有 n(n-1)/2 条潜在沟通关系（5 人 10 条，10 人 45 条）；“我的部分已经好了”的四重陷阱——目标不一致、依赖在等待、完成的定义不同、责任落在缝隙里[20]

幻灯片把“我的部分已经好了”拆成四重陷阱：**目标不一致**——你要赶紧上线，我要趁机重构，两个人都觉得自己在帮项目；**依赖在等待**——前端等接口，后端等需求，每个人都很忙，关键路径却没动；**完成的定义不同**——我的代码能跑，你以为已经能交付，一集成才发现各自理解不同；**责任落在缝隙里**——接口、测试、部署都“有人负责”，跨模块的问题却没人认领。就算你能把分内的事做好，因为平方级的沟通关系，你不完全知道别人在做什么，最后就会各种冲突：你写的测试我过不了。

因此现在软件工程团队里最难招的不是码农，是能把任务分解的 **team leader**：一个 100 人的团队能分出 10 个小团队，10 个 team leader 各带一摊，最后拼起来还是一个产品——这样的人最稀缺；能带小团队的人次之。

7.3 版本控制留给我们的启发

如果把这套管理问题交给未来的你去当主 Agent，版本控制留下了什么启发？幻灯片的答案是：**项目可以不仅是“当前状态”**。

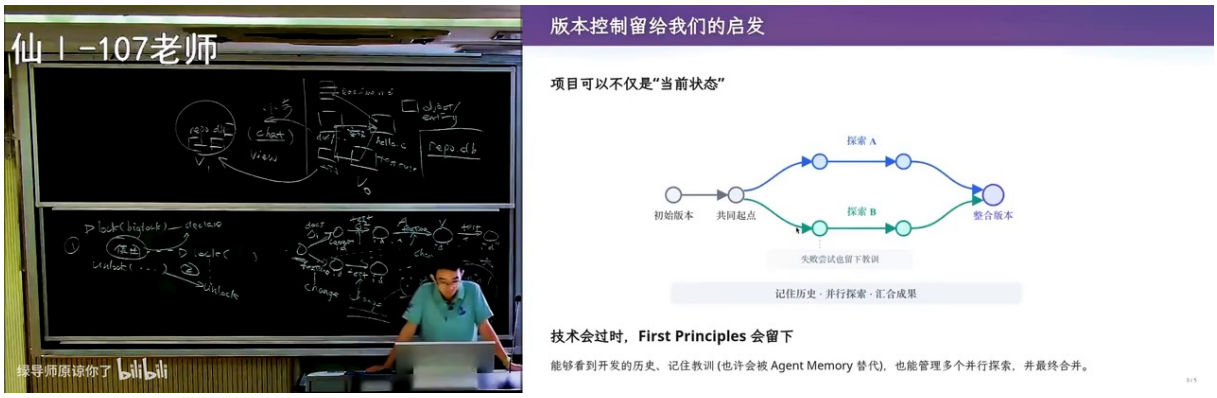


图 21: 幻灯片“版本控制留给我们的启发”：树状图画出初始版本、共同起点、探索 A、探索 B、整合版本——“失败尝试也留下教训”；“记住历史 · 并行探索 · 汇合成果”；技术会过时，First Principles 会留下[21]

[20] 视频画面时间区间：01:05:33–01:05:48。若为裁剪图，时间区间仍对应原视频画面。
[21] 视频画面时间区间：01:06:48–01:07:03。若为裁剪图，时间区间仍对应原视频画面。

能够看到开发的历史、记住教训（也许会被 Agent memory 替代），也能管理多个并行探索（探索 A、探索 B），并最终合并成整合版本。失败尝试也留下教训——它们不是垃圾，是未来绕开坑的路标。讲师的总结是：技术会过时，First Principles 会留下。

7.4 本章小结

本章回答“工具之上还剩什么”。版本控制是工具，真正的困难是进度管理：沟通关系随人数平方级上升（5 人 10 条、10 人 45 条），最难招的是能把任务分解的 team leader；AI 的斩杀线暂时没到这一层，摩尔定律还在推动模型变强。版本控制留下的启发是：项目不只是当前状态——记住历史、并行探索、汇合成果，失败也留下教训。下一章把这些准则落到“从零开始一个项目”的现场演示里。

8 从零开始一个项目：以不变应万变

8.1 你就是包工头：从零开始的难度

项目的早期阶段你肯定不能用 multi-agent；你就是**包工头**：先找一个你能用到的最强的 Agent，跟它多轮对话。任何时候，开始都比不开始强，所以讲师决定现场直接拿 Lab 开干：打开课程作业页面，把需求交给 Agent，完全没有事先准备过。

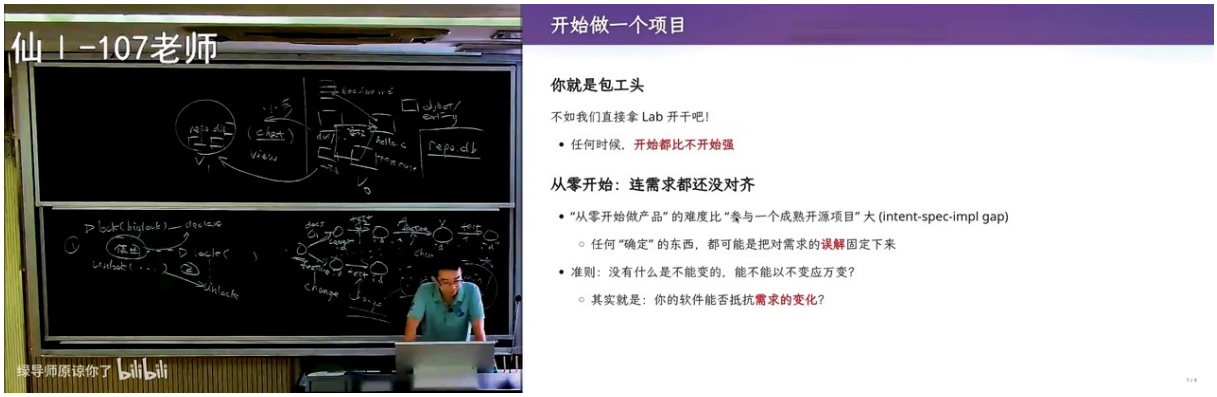


图 22: 幻灯片“开始做一个项目”：你就是包工头，不如直接拿 Lab 开干；任何时候开始都比不开始强；从零开始连需求都还没对齐——“从零开始做产品”的难度比“参与一个成熟开源项目”大（intent-spec-impl gap）[22]

这里有一个重要的不对称：“从零开始做产品”的难度比“参与一个成熟开源项目”大得多，因为**连需求都还没对齐**（intent-spec-impl gap）。而且任何“确定”的东西，都可能是把对需求的误解固定下来。讲师给出的准则是：没有什么是不能变的——能不能以不变应万变？其实就是一句话：你的软件能否抵抗需求发生根本性的变化？这恰恰是软件工程课不教、数据库课和操作系统课也不教的东西：它们告诉你“这个系统是长这样的”，而软件工程说的是“对未来需求的变化”。

8.2 一个岔开的话题：汉诺塔与 Pico Playground

在演示之前，讲师岔开讲了一个他看到的题目（据说是小学生考级题）：汉诺塔（Hanoi）人人都会，但加一个额外要求——**所有盘子的移动只能往一个方向**。三个盘子的时候就能看出问题：正

[22] 视频画面时间区间：01:18:03–01:18:18。若为裁剪图，时间区间仍对应原视频画面。

常解法先把两个移到中间，但单向限制下你做不到，必须重新设计移动方案。最优解的思路是：最大的盘子总是“向右移一格、再向右移一格”，剩下的用归纳法（induction）推出也是最优的。

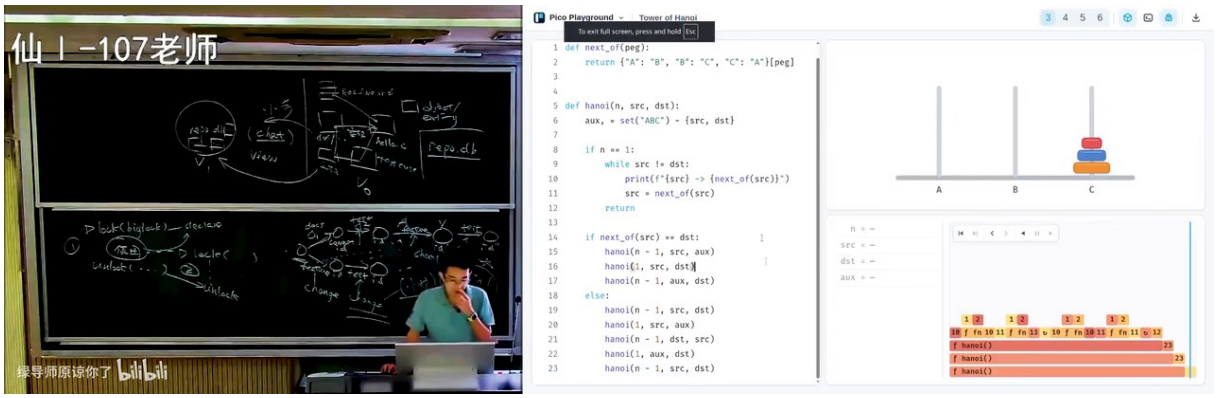


图 23: 讲师现场演示的 Pico Playground《Tower of Hanoi》：右侧代码编辑器里的 `next_of` 与 `hanoi` 实现（`aux = set("ABC") - {src, dst}`、`while src != dst: ...`），可视化随代码运动[23]

他为什么岔开讲这个？因为他准备了一个小产品：Pico Playground，一个可以编辑代码并实时看汉诺塔可视化的 playground。可视化随代码运动：写错了（比如试图把小的移到大的上），就能在可视化里定位到那条语句。要把这个产品卖给小学生挣钱，当然是提供付费增值服务——AI 助教：小朋友实在做不下去时点一个“我要助教”，DeepSeek 的 token 就开始燃烧，而他的 prompt 是独门的，作为有经验的老教师，他可以敏锐地知道小朋友的问题在哪里，提供个性化指导，并收取比 DeepSeek 原价更贵的 token 费记在账户上。

8.3 GPT 的“正常”设计 vs 版本管理

这就涉及要做一个信息系统了。讲师让学生来设计，学生和 AI 聊完之后，GPT 给出的原话是：“一个 API 应用，一个关系数据库，一个对象存储；需要异步任务时再加后台任务进程；数据库关系和记录、大文件与完整轨迹存对象存储，任务量小时数据库任务表就够用。”这是一个非常正常的、你们上软件工程课会给出的设计。但讲师**不喜欢这个设计**。

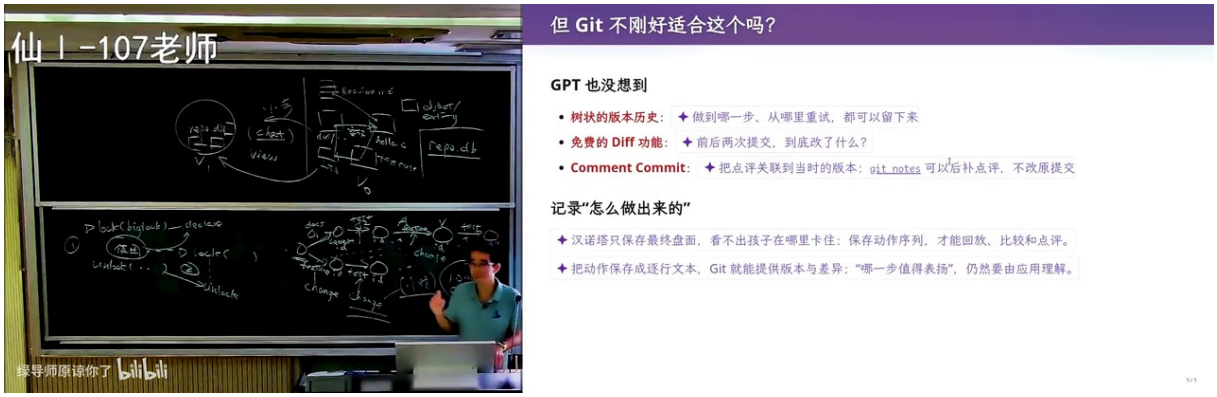


图 24: 幻灯片“但 Git 不刚好适合这个吗?”：GPT 也没想到——树状的版本历史（做到哪一步、从哪里重试都可以留下来）、免费的 diff 功能（前后两次提交到底改了什么）、Comment Commit（把点评关联到当时的版本，git notes 可以后补点评不改原提交）；汉诺塔只保存最终盘面看不出孩子在哪里卡住，保存动作序列才能回放、比较和点评[24]

[23] 视频画面时间区间：01:11:33–01:11:48。若为裁剪图，时间区间仍对应原视频画面。

他的理由是从 Overleaf 说起的：Overleaf 有一个功能，可以把协作文档当成一个 Git 仓库导出。他最喜欢这个功能，因为给学生改论文时，可以直接把论文克隆到本地，不用忍受网页界面，打开 Codex 让它逐个`.tex`文件修复语法错误，论文就改好了。同理，这个 playground 难道不应该在学生每改一次、每次点运行或切换时**自动做一个 Git 提交**吗？申请 AI 助教时它转圈圈，点评就可以在 Git 提交上产生一个分叉（comment commit）。GPT 没有算到的是：产生一个 commit 的成本**远远小于**把剩下的东西 persist 到数据库的成本，后者几乎可以忽略不计；用数据库确实有好处（总体压缩更小、性能更高），但这不是他要的——他的本质需求是**版本管理**。所以最早期他可以把小朋友的代码直接存在自己的代码库里，在上面创建 Git 历史记录；直到文件目录格式、commit message 格式、文件使用方式都收敛了，再迁移到数据库也不迟。

8.4 现场演示：直接丢需求会得到什么

回到现场演示。有两种做法：第一种是直接把实验要求粘进去回车，每个同学都可以体验一下；讲师替大家体验，没告诉会发生什么。Agent 很快就开始写代码——“只能用魔法打败魔法，这个每秒 **300** token 的生成速度实在是太恐怖了”。

提示词工程最危险的一点：prompt 舔狗

你提示词里的每一个词，self-attention 都会看到。AI 已经被训练成了 prompt 的“舔狗”，有一种无法往外拉的、必须遵循你指令的倾向。所以当你把一个讨论稿式的开放需求丢给它时，它会**过度遵循**指令，最终生成一个“确实满足了指令，但其实不是你想要的”的东西。实验要求里最重要的是那句“实现一个持续为你工作的个人助手”，其他的话都只是解读。

结果验证了这一点：Agent 生成的东西从 README 来看每一步都“正确”，但最终得到的是一个你不想要的东西——它做了一个教务系统式的业务逻辑系统，有入口层、编排层、能力层、数据层，看起来已经像一个教务系统了；而它甚至都没有创建一个 Git 仓库。讲师想要的学习助手（自动从外网收集数据、层次化归档、给一个好界面）十有八九没有。这就是**你的 intent / specification 和它的理解之间的巨大 gap**：但当模型看到你具体说“要实现这个”时，它就真的去实现了。

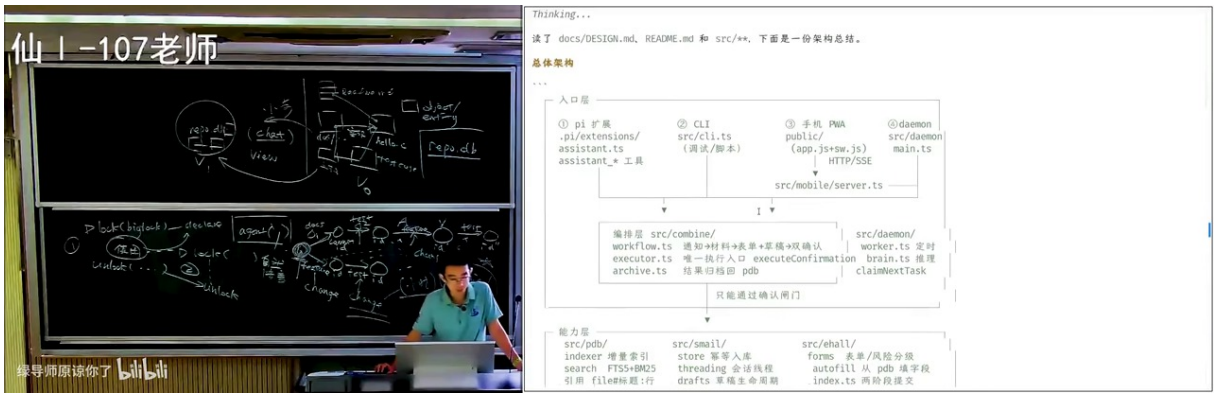


图 25: Agent 生成的“架构总结”：入口层（pi 扩展 / CLI daemon / 手机 PWA）、编排层、能力层、数据层——一个非常典型的分层架构，但它把需求当成了业务逻辑系统来做[25]

这和古法编程形成对照：如果是一个成熟的产品经理，老板提出需求后，他肯定会先跟团队沟

[24] 视频画面时间区间：01:15:03–01:15:18。若为裁剪图，时间区间仍对应原视频画面。

[25] 视频画面时间区间：01:32:03–01:32:18。若为裁剪图，时间区间仍对应原视频画面。

通——项目怎么管、每天什么时候开会——甚至在写下第一行代码之前。Agent 很好玩，它直接上手就做，觉得这是一个“用一次就丢掉”的东西。

8.5 MVP：先验证最值得验证的假设

如果真的要从零开始构建，更符合主流 best practice 的叫 **MVP**（minimum viable product），有一个最小的验证版本。有一句话：**the biggest risk in product development is building something nobody wants**——你做的东西没人要，这是最大的风险。刚才那个 Agent 做的东西，绝大部分不是讲师要的。



图 26: 幻灯片“用 MVP 验证需求”: Minimum Viable Product 用一个最小的版本验证; “The biggest risk in product development is building something nobody wants.”; MVP 会给你一个立即能使用、不完备的版本，帮助你考虑未来潜在的变化[26]

MVP 逼迫你想清楚：什么样的需求才是你**绝对要的**。对讲师来说，个人助手最核心的是“提升学习效率的能力”和“提升工作效率的能力”。总结出这两条之后，第一个 MVP 里可以**没有任何数据库**：因为任何时候文件系统膨胀了，你都可以跟 Agent 说“按照这个数据的组织帮我提炼出一个数据库”，这个问题就结束了。反过来的风险是：一旦一开始就带上数据库，之后每次重构都需要 Agent 做数据库迁移；而文件系统不一样——你自己就可以在文件管理器里调整目录结构，逐渐梳理到舒服为止。看看你调文件和调 database schema 的心智负担：人类从小学电脑课开始就操作文件目录，对那个模式非常习惯；而面对一个未知的 schema，你需要强行做一次概念的翻译。

讲师自己的回邮件 Agent 就没有用任何数据库。他的流程是：Agent 收邮箱，把 email id 放到共享云盘同步上；id 里带着 Agent 解析出来的回复草稿、邮件分类（是不是垃圾邮件）；再由下一级流水线处理。先要明白的是系统里有哪几级流水线、它们之间怎么协作、哪些东西通过云盘同步。后来他提出一个新的需求变更——“我希望邮件、资料等都和云盘保持同步”，然后让 Agent 帮他看一看架构做得好不好：什么是一个好的软件架构？当前实现可能满足当前需求，但当需求发生变更时，你做出的变化是摧毁性的、还是背上技术债、还是几乎不动，那就见分晓了。

[26] 视频画面时间区间：01:32:33–01:32:48。若为裁剪图，时间区间仍对应原视频画面。

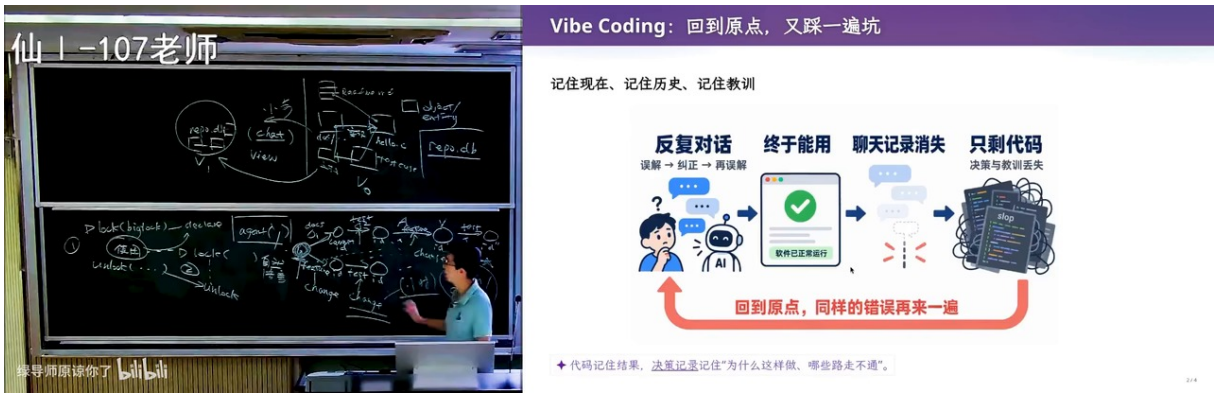


图 27: 幻灯片“Vibe Coding：回到原点，又踩一遍坑”：反复对话 → 终于能用 → 聊天记录消失、只剩代码 → 误解 → 纠正 → 再误解 → 决策与教训丢失 → 软件已正常运行 → 回到原点，同样的错误再来一遍；代码记住结果，决策记录记住“为什么这样做、哪些路走不通”[27]

这就是 Vibe Coding 的“回到原点，又踩一遍坑”：反复对话终于能用，聊天记录消失、只剩代码；误解 → 纠正 → 再误解，决策与教训丢失；软件已经正常运行，下一个需求来了又回到原点，同样的错误再来一遍。幻灯片给出的解法是那句话：**版本管理做的三件事——记住现在、记住历史、记住教训**。这在 Agent 时代和古法编程时代都成立：记住现在，是因为你必须要有一个 repository，不管是 C/C++、Python 还是 JavaScript 项目，都要把文件管起来；记住历史，是为了有一天这个东西不要了，你还能把它捞回来，还能看到你是怎么演进到某个点然后发现失败的；教训会不停地警示你下一步该怎么做。代码记住结果，决策记录记住“为什么这样做、哪些路走不通”。

8.6 早期的 Prompt，影响最大

最后一个幻灯片把整场演示压缩成一句话：每一次“确定”都在影响项目的走向，而早期的 prompt 影响最大。

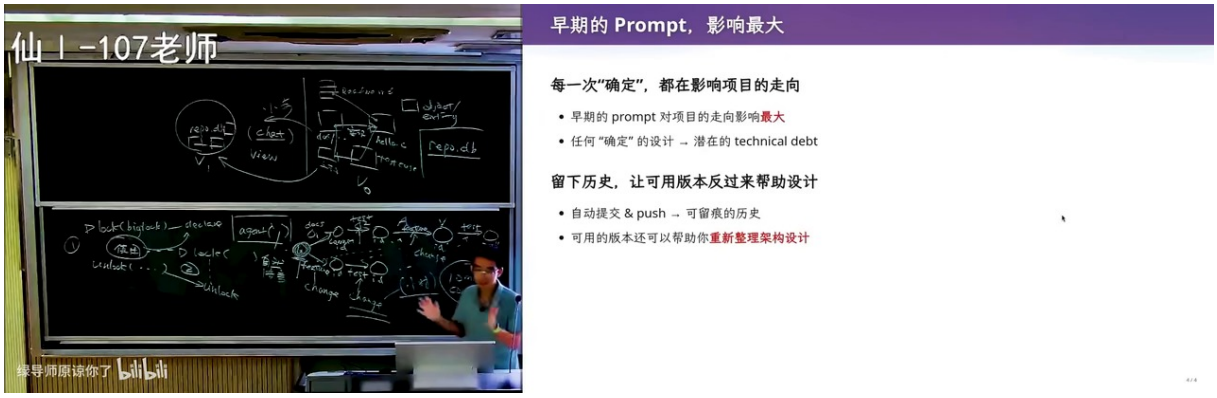


图 28: 幻灯片“早期的 Prompt，影响最大”：每一次“确定”都在影响项目的走向；任何“确定”的设计都有潜在的 technical debt；留下历史，让可用版本反过来帮助设计[28]

这一演示还意味着：**你早期的 prompt 对整个项目走向的影响是最大的**。你最早把 README 给它、说“我要做一个 MVP”，这些都会对项目造成很大影响。所以回到项目、回到管理：不能直接把任务丢给 Agent，第一件事一定是创建一个空的仓库；然后想清楚下一步要迈向哪里，往前再迈

[27] 视频画面时间区间：01:29:33–01:29:48。若为裁剪图，时间区间仍对应原视频画面。
[28] 视频画面时间区间：01:38:03–01:38:18。若为裁剪图，时间区间仍对应原视频画面。

一小步。留下历史，让可用的版本反过来帮助设计；任何“确定”的设计都可能是潜在的 technical debt。

8.7 本章小结

本章回答“从零开始怎么干”。从零做产品比参与成熟项目更难，因为需求还没对齐；直接把开放需求丢给 Agent 会得到一个“满足指令但不是你想要的”的东西（教务系统式的分层架构，连 Git 仓库都没有建）。正确姿势是：第一件事建空仓库，用最强 Agent 多轮对话，用 MVP 验证最值得验证的假设；早期不要上数据库，文件系统优先；记住版本管理三件事——记住现在、记住历史、记住教训；并意识到早期的 prompt 对项目走向影响最大。

9 总结与延伸

9.1 讲者的收尾

讲师在收尾时把两节课的版本管理串成了一条线：从历史动机（文件系统是 1950 年代的将错就错）到 Git 的快照模型，到把 change 变成一等公民的 Jujutsu，再到 Agent 时代用悲观锁重新设计并发控制；最后落到从零开始一个项目的现场演示——直接丢需求会得到 slop，正确姿势是建空仓库、多轮对话、MVP 先行、文件系统优先。下一讲（周日调休课）进入**软件架构与构造**的内容。

9.2 本讲义的提炼

把全讲压缩成几条可以带走的准则：

- **限制塑造工具**：stash、branch、每日例会、blame，都是“人是以小时为单位产生 change 的 single-threaded 生物”这一限制的产物。Agent 以秒级产生原子提交、还能 10 倍 100 倍并行时，这些设计的适用性要重新审视。
- **先问“我们真正想操作什么”**：Git 用 commit 代替 change，rebase 让同一次修改失去身份；把 change 变成一等公民（change ID 延续、冲突可保存）就得到了 Jujutsu。设计新系统时，先找出现有工具里“用替代品凑合”的概念。
- **并发控制可以换方向**：Git 近似乐观并发控制；Agent 快 1000 倍之后冲突频繁，改用悲观锁（上锁 → 读取最新状态 → 修改 → 验证 → 解锁），主 Agent 规划、子 Agent 上锁开干，而且锁不会自动让错误的修改变正确。
- **历史的价值转向**：Agent 是同模型克隆体、不能被开除，blame 不再必要；历史从“谁写错了”转向“哪次变化破坏了什么、怎样恢复”。快照仍然要保留，因为走错方向时要能回退。
- **需求与架构的关系是本讲的主轴**：从 Lab 1 的五项能力、到 repo.db 的实体-关系图、到 GPT 的数据库设计被 Git 替代、到第一个 MVP 不带数据库——反复出现的准则都是“以不变应万变”：架构能否用最小的变化抵抗需求的根本变化。
- **对需求的态度**：提示词里的每个词都会被 self-attention 看到，AI 会过度遵循你的指令；最重要的一句是“实现一个持续为你工作的个人助手”，其余是解读。先想清楚什么是你绝对要的，再让 Agent 动手。

这些方案（repo.db、悲观锁、把 change 作为一等公民）来自讲者的推演与课堂现场实验，还没有经过大规模工程实践的检验；能带走的不是结论清单，而是“先问清楚这个工具在替谁做假设”的方法。本讲结束后可以直接追问的具体问题是：一个从零开始的项目，架构应该怎样设计才能在需求变化时只做最小改动？这正是下一讲（软件架构与构造）要回答的内容。

一句话总结

版本管理做的三件事——记住现在、记住历史、记住教训——在 Agent 时代依然成立；变的是“谁在产生 change”：当 change 从小时级变成秒级、从一个人变成一群克隆体，版本管理就要从“为人设计”转向“为 Agent 设计”，而工具之上的团队管理与需求对齐，才是真正困难的部分。

9.3 拓展阅读

- Jujutsu（jj）版本控制系统：与 Git 兼容、以 change 为一等公民、无需 staging area，官方 tutorial 一开始就介绍这些设计动机。
- `git-worktree` 文档：一个仓库多个工作目录，worktree 里的`.git` 是指向`.git/worktrees/` 的文本文件。
- P. Mader & A. Zisman, *Software traceability: trends and future directions*, FOSE 2014, pp. 55–69：软件可追踪性的综述。
- F. Brooks, *The Mythical Man-Month*（人月神话）：沟通关系平方级上升与团队管理的经典教训。
- E. Ries, *The Lean Startup*：MVP（minimum viable product）与“验证最值得验证的假设”。